

LangGraph Cheatsheet

Command & Send API, multi-agent mimarileri, time-travel, production & deployment.

langgraph.types · langgraph.pregel · langgraph-turkce.github.io

1. Command & Send API

Command — yönlendirme + güncelleme

```
from langgraph.types import Command

def router(state) -> Command:
    return Command(
        goto="arastirmaci" if ... else "yazar",
        update={"not": "..."}
    )
```

Send — dinamik paralel dağıtım

```
from langgraph.types import Send

def dagit(state):
    return [Send("ozetle", {"belge":b})
            for b in state["belgeler"]]
```

Çalışma zamanında kaç dal açılacağı belirsizken (map-reduce) kullanılır.

2. Multi-Agent Mimarileri

Supervisor deseni

```
g.add_node("supervisor", supervisor_node)
g.add_node("arastirmaci", arastirmaci_agent)
g.add_conditional_edges("supervisor", route_next)
```

Merkezi karar → yüksek denetlenebilirlik.

Supervisor vs. Swarm

- **Supervisor:** denetlenebilir, kurumsal iş akışları
- **Swarm:** ajanlar arası doğrudan devir (handoff), esnek ama takibi zor
- **Hiyerarşik:** subgraph'larla iç içe supervisor'lar

3. Time Travel & Debug

State geçmişi

```
for s in app.get_state_history(config):
    print(s.values, s.next)
```

Belirli node'dan devam

```
app.update_state(config, {...},
                 as_node="chatbot")
```

Önceki bir adıma dönüp farklı bir dal deneyebilirsiniz.

4. Production & Deployment

Retry politikası

```
from langgraph.pregel import RetryPolicy

g.add_node("arac_cagir", node,
           retry=RetryPolicy(max_attempts=3))
```

Recursion limit

```
app.invoke(input,
           config={"recursion_limit": 25})
```

Döngüsel graflarda sonsuz döngü / maliyet patlamasını önler.

5. langgraph.json & CLI

Proje manifesti

```
{
  "graphs": {"bot": "./src/graph.py:app"},
  "dependencies": [".", "."],
  "env": ".env"
}
```

langgraph.json — LangGraph Platform'un projeyi nasıl çalıştıracaklarını tanımlar.

CLI komutları

```
pip install langgraph-cli

langgraph dev # yerel + görsel debug (Studio)
langgraph build # Docker image oluştur
langgraph up # yerel deploy
```

6. Proje Yapısı & Sık Hatalar

Klasör yapısı (özet)

```
src/bot/  
  graph.py      # StateGraph + compile()  
  state.py      # State şeması  
  nodes/        # her node ayrı dosya  
  tools.py  
  tests/
```

Sık hatalar

- **GRAPH_RECURSION_LIMIT** — döngü END'e ulaşmıyor
- **MISSING_CHECKPOINTER** — interrupt() için checkpointer eksik
- **INVALID_CONCURRENT_GRAPH_UPDATE** — paralel yazımda reducer eksik

7. Senior Kontrol Listesi

- **Checkpointer prod'da kalıcı mı?** (Postgres/Redis, MemorySaver değil)
- **Observability gün 1'den kurulu mu?** (LangSmith/Langfuse trace'leri node bazında etiketli)
- **recursion_limit ve max token bütçesi tanımlı mı?**
- **Kritik/geri alınamaz aksiyonlar için human-in-the-loop var mı?**
- **Supervisor mi swarm mi:** denetlenebilirlik mi esneklik mi öncelikli?

8. Hızlı Referans

KAVRAM	NE İŞE YARAR
Command	Yönlendirme + state güncelleme
Send	Dinamik paralel dağıtım (map-reduce)
get_state_history	Time-travel (zamanda geri gitme) / debug
RetryPolicy	Node bazlı yeniden deneme (retry)
recursion_limit	Döngü üst sınırı
langgraph.json	Deployment manifesti
langgraph dev/build/up	CLI komutları

Sık yapılan hata: recursion_limit koymadan döngü içeren bir grafi production'a almak — mantık hatasında maliyet kontrolsüz artar.

LangGraph Türkçe Rehber · Bağımsız, resmi olmayan eğitim kaynağı · LangGraph MIT lisanslıdır (langchain-ai) · Önceki: Orta Seviye · Uçtan uca örnek proje: langgraph-turkce.github.io/ornek-proje